

Spécifications techniques

Menu Maker - Qwenta

Version	Auteur	Date	Approbation
1.0	Floane VARATHEN	26/06/2026	Soufiane

I. Choix technologiques	2
II. Liens avec le back-end	7
Quel langage pour le serveur	7
A-t-on besoin d'une API ? Si oui laquelle ?	8
Base de données choisie	10
III. Préconisations concernant le domaine et l'hébergement	10
Nom de domaine	10
Hébergement	11
Adresse email	11
IV. Accessibilité	12
Compatibilité navigateur	12
Types d'appareils	12
Conformité WCAG	12

V. Recommandations en termes de sécurité	13
Accès aux comptes	13
Plugins et dépendances	14
VI. Maintenance du site et futures mises à jour	15
Mises à jour et dépendances	15
Sauvegardes	15
Déploiement continu	15
Evolutions futures	15

I. Choix technologiques

- État des lieux des besoins fonctionnels et de leurs solutions techniques :

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
Création de l'interface utilisateur	L'interface doit être dynamique, réactive et composée de nombreux éléments interactifs (formulaires, modales,	React 19.2	Bibliothèque JavaScript open source développée par Meta, basé sur des composants réutilisables et un DOM virtuel pour des mises à jours performantes	1) La nature dynamique de l'éditeur de menu (ajout/suppression des plats en temps réel) correspond au modèle de composants React. 2) React est la

	prévisualisation en temps réel)			bibliothèque front-end la plus utilisée au monde, garantissant un large écosystème et une communauté active pour la maintenance.
Navigation entre les pages de l'application	L'application est une SPA (Single Page Application) : la navigation ne doit pas recharger la page	React Router v8.1	Bibliothèque de routing standard pour React, permettant de définir des routes déclaratives et de gérer la navigation côté client	1) Solution de routing officielle et la plus adaptée dans l'écosystème React alignée sur React 19+. 2) Permet de protéger les routes privées (dashboard) derrière une vérification d'authentification JWT.
Gestion des formulaires (inscription, connexion, création de menu)	Les formulaires doivent être validés côté client avant envoi au serveur, avec des messages d'erreur clairs	React Hook Form + Zod	React Hook Form gère l'état des formulaires sans re-render inutile ; Zod définit des schémas de validation typés et réutilisables	1) React Hook Form est plus performant que Formik (moins de re-render) pour des formulaires complexes comme la création de menu. 2) Zod permet de partager les mêmes schémas de

				validation entre le front-end et le back-end Node.js , réduisant la duplication de code.
Mise en page et styles de l'interface	Le design doit respecter la charte graphique Qwenta (couleurs, typographies) et être maintenable par l'équipe	Tailwind CSS (structure et UI fixe uniquement)	Framework CSS utilitaire permettant de styler directement dans le JSX via des classes prédéfinies, utilisé pour le dashboard, les formulaires et la navigation	1) Tailwind évite la création de feuilles CSS volumineuses, chaque composant portant ses propres styles. 2) La personnalisation via tailwind.config.js permet d'intégrer facilement la palette couleur Qwanta sur l'interface fixe de l'outil.
Personnalisation dynamique du menu (police, couleur, logo)	Le restaurateur doit pouvoir choisir dynamiquement la police et la couleur de son menu	Variables CSS natives	Les valeurs choisies par l'utilisateur sont injectées en variables CSS appliquées au composant de rendu du menu, en complément de Tailwind	1) Tailwind génère ses classes au build et ne peut pas refléter une valeur saisie à l'exécution ; les variables CSS sont modifiables en JavaScript à tout moment. 2) Solution légère, native au navigateur, sans dépendance supplémentaire.

Upload et gestion du logo restaurant	Le restaurateur doit pouvoir téléverser et prévisualiser son logo ; le fichier doit être validé, sécurisé côté serveur, et optimisé avant stockage pour ne pas alourdir le chargement du menu en ligne	react-dropzone (front) + Multer (back) + Sharp (back)	react-dropzone gère la zone de dépôt/sélection et la prévisualisation du fichier côté navigateur, avec une visualisation Zod du type et du poids avant envoi. Multer middleware Express, réceptionne le fichier envoyé en multipart/form-data côté serveur et applique une validation de sécurité (type MIME réel, taille maximale). Sharp redimensionne et compresse ensuite l'image (conversion en WebP) avant son stockage définitif.	1) La validation est faite à deux niveaux (front avec Zod pour le confort utilisateur, back avec Multer pour la sécurité réelle, un fichier ne devant jamais être considéré fiable uniquement parce qu'il a été validé côté client). 2) Sharp réduit significativement le poids du logo stocké et servi, ce qui améliore le temps de chargement du menu en ligne consulté par les clients du restaurant sans dépendance lourde.
Export des menus en PDF	Les restaurateurs doivent pouvoir télécharger leur menu en PDF depuis le navigateur	@react-pdf/renderer	Génère un PDF vectoriel à partir de composant React, avec texte réé sélectionnable	1) Contrairement à une solution basée sur la capture d'écran (image), le texte reste sélectionnable et lisible par les outils d'accessibilité, cohérent avec

				l'exigence WCAG AA. 2) Cohérent avec la stack React : les mêmes composants peuvent piloter à la fois l'aperçu à l'écran et le PDF généré.
Création de modales (ajout de catégorie, confirmation de suppression)	Les modales doivent être accessibles aux claviers et compatibles avec les lecteurs d'écran	Radix UI Dialog	Composant de modale accessible, gérant nativement le focus trap et les attributs ARIA	1) Gestion automatique du focus trap et des attributs ARIA, une exigence WCAG AA, avec une maintenance active. 2) Compatible avec Tailwind pour le style, cohérent avec le reste de la stack front-end.
Gestion des requêtes HTTP vers l'API	Le front-end doit communiquer de façon fiable avec le back-end, avec gestion des erreurs, du cache et de la session JWT	Axios + TanStack Query, avec withCredentials: true	Axios assure le transport HTTP des requêtes vers l'API Express ; TanStack Query gère le cache, l'invalidation et les états loading/error au-dessus	1) Le JWT étant stocké en cookie httpOnly, il n'est jamais accessible en JavaScript : c'est le navigateur qui l'envoie automatiquement à chaque requête vers le bon domaine, à condition que withCredentials soit activé côté client et

				que le back-end Express configure CORS avec credentials: true et une origin explicite (Vercel). 2) TanStack Query évite de réécrire la gestion des états de chargement et d'erreur pour chaque appel API, notamment lors de la création/modification d'un menu.
--	--	--	--	---

II. Liens avec le back-end

Quel langage pour le serveur

Le serveur est développé en [Node.js](#) avec le framework [Express.js](#)

[Node.js](#) a été retenu pour deux raisons principales :

- Il utilise le même langage JavaScript que le front-end React, ce qui facilite le partage de code (notamment les schémas de validation Zod)
- Il réduit le coût de montée en compétences pour l'équipe. [Express.js](#) est le framework [Node.js](#) le plus utilisé, léger et non opiniâtre, ce qui laisse la liberté d'organiser l'architecture selon les besoins du projet.

A-t-on besoin d'une API ? Si oui laquelle ?

Oui l'application repose sur une API REST développée en interne avec [Express.js](#). Les principaux endpoints sont :

Méthode	Route	Description
POST	/api/auth/login	Inscription / connexion et envoie d'un magic link à l'adresse email saisie (Resend).
GET	/api/auth/verify	Valide le token du magic link reçu par email et création du JWT en cookie httpOnly
POST	/api/auth/logout	Déconnexion et suppression du cookie httpOnly côté serveur
GET	/api/menus	Récupération de tous les menus de l'utilisateur connecté
POST	/api/menus	Création d'un nouveau menu
GET	/api/menus/:menuId	Récupération d'un menu existant
PUT	/api/menus/:menuId	Modification d'un menu existant (nom)
DELETE	/api/menus/:menuId	Suppression d'un menu existant
POST	/api/menus/:menuId/categories	Ajout d'une catégorie à un menu existant
PUT	/api/menus/:menuId/categories/:catId	Modification d'une catégorie existante (nom)

DELETE	/api/menus/:menuId/categories/:catId	Suppression d'une catégorie
POST	/api/menus/:menuId/categories/:catId/plats	Ajout d'un plat (nom, prix, description, image via Multer + Sharp)
PUT	/api/menus/:menuId/categories/:catId/plats/:platId	Modification d'un plat existant
DELETE	/api/menus/:menuId/categories/:catId/plats/:platId	Suppression d'un plat
PUT	/api/menus/:menuId/style	Modification dy style d'un menu existant
GET	/api/restaurant	Récupération des informations du restaurant
PUT	/api/restaurant	Modification des informations du restaurant
POST	/api/restaurant/logo	Upload du logo (Multer + Sharp)
GET	/api/auth/account	Récupération des informations du compte
PUT	/api/auth/account	Modification des informations du compte
POST	/api/auth/account/email	Lier un email au compte
DELETE	/api/auth/account/email/:emailId	Délrier un email du compte
GET	/api/auth/branding	Récupération des informations du restaurant

POST	/api/auth/branding/logo	Ajout du logo
PUT	/api/auth/branding/logo	Remplacement du logo
DELETE	/api/auth/branding/logo	Suppression du logo
PUT	/api/auth/branding/colors	Ajout des couleurs de branding
DELETE	/api/auth/branding/colors	Suppression des couleurs de branding
GET	/public/menus/:shareToken	Récupération du lien public au menu sans authentification (Devloveroo et Instagram)
GET	/api/blog/articles?limit=3	Récupération des 3 derniers articles du blog Qwenta

Base de données choisie

PostgreSQL : base de données relationnelle SQL

PostgreSQL est adapté aux données structurées : un utilisateur possède plusieurs menus, chaque menu contient plusieurs catégories, chaque catégorie contient plusieurs plats. Ce modèle relationnel est naturellement exprimé en SQL. Prisma garantit des requêtes paramétrées (protection contre les injections SQL) et facilite les migrations de schémas.

III. Préconisations concernant le domaine et l'hébergement

Nom de domaine

Le site sera accessible via un sous-domaine Qwenta, dont la configuration est en cours de validation côté client (ex. menumaker.qwenta.fr). Le certificat SSL/TLS sera mis en dès l'ouverture du domaine pour garantir des connexions HTTPS.

Hébergement

Couche	Hébergeur	Justification
Front-end React	Railway	Déploiement automatique depuis GitHub à chaque push sur main. Héberger le front sur la même plateforme que le back élimine toute configuration CORS cross-origin et permet d'utiliser un cookie JWT en SameSite=Strict (plus sécurisé).
Back-end (Node.js + Express)	Railway	Supporte nativement Node.js , interface simple de gestion des variables d'environnement, déploiement automatique depuis GitHub.
Base de données (PostgreSQL)	Railway	Base de données hébergée dans le même environnement que le back-end, ce qui minimise la latence des requêtes et simplifie la gestion des accès (pas de configuration réseau inter-hébergeurs).

Adresse email

Les adresses email professionnelles seront configurées sur le domaine Qwenta validé (ex. contact@qwenta.fr).

Le service d'envoi transactionnel retenu est Resend pour les emails automatiques (confirmation de connexion) pour les raisons suivantes :

- Son API et son SDK [Node.js](#) officiel sont conçus pour une intégration simple avec une configuration minimale.
- Son tier gratuit (3 000 emails/mois) est largement suffisant pour la phase de lancement, sans les contraintes de validation de domaine complexes de SendGrid.

IV. Accessibilité

Compatibilité navigateur

Menu Maker est compatible avec les trois dernières versions des navigateurs suivants :

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
- Opera

Types d'appareils

L'application est conçue pour une utilisation desktop uniquement, conformément aux spécifications fonctionnelles. Aucune version mobile ou tablette n'est prévue pour ce lancement. La résolution minimale supportée est 1280 x 720 px. Ce qui correspond à la résolution native des laptops professionnels. Une largeur maximale de contenu de 1440 px est appliquée côté CSS pour garantir un affichage confortable sur les très grands écrans sans étirer le layout.

Conformité WCAG

L'application vise la conformité WCAG 2.1 niveau AA. Les mesures concrètes mises en oeuvre sont :

Critère	Mise en oeuvre
Navigation clavier	Tous les éléments interactifs (boutons, liens, champs) sont accessibles via Tab/Entrée/Echap, avec un focus visible à tout moment

Lecteurs d'écran	Attributs aria-label, aria-describedby et rôles ARIA sur tous les composants React interactifs
Contraste des couleurs	Ratios vérifiés via Colours (objectif AA : 4,5:1 texte normal, 3:1 texte large et composants). Écart identifié sur les boutons de la charte Qwenta ; ajustement des teintes à proposer au client.
Formulaires	Chaque champ associé à un <label> explicite ; les erreurs de validation sont annoncées via aria-live
Images	Attribut alt systématique sur toutes les balises ; images décoratives marquées alt=""
Modales	Gestion du focus trap (focus trap) via Radix UI Dialog ; fermeture par la touche Echap

V. Recommandations en termes de sécurité

Accès aux comptes

Risque	Contre-mesure
Vol de session (XSS)	JWT stocké en cookie httpOnly : inaccessible depuis JavaScript protège contre les attaques XSS qui tenteraient de lire le token.
Brute force sur l'authentification	Rate limiting sur les routes /api/auth/* via le middleware express-rate-limit (ex. 5 tentatives maximum par

	tranche de 15 minutes par IP).
Mot de passe compromis	Hachage des mots de passe avec bcrypt avant stockage en base. Aucun mot de passe en clair.
Injections SQL	Toutes les requêtes passent par Prisma, qui utilise des requêtes paramétrées. Aucune concaténation de chaînes SQL.
Données non validées côté serveur	Validation systématique des entrées côté back-end avec Zod, indépendamment de la validation front-end.
Upload de fichiers malveillants	Multer vérifie le MIME réel du fichier (pas uniquement l'extension) et impose une taille maximale avant tout traitement.
Communications non chiffrées	HTTPS obligatoire sur tous les services Railway (certificat SSL automatique). Cookie JWT configuré avec l'attribut Secure.
Dépendances vulnérables	Audit régulier des dépendances via npm audit à chaque déploiement. Politique de mise à jour mensuelle des packages.
Variables sensibles exposées	Toutes les clés d'API, secrets JWT et credentials de base de données sont stockés en variables d'environnement Railway, jamais dans le code source.

Plugins et dépendances

Les dépendances npm sont auditées à chaque déploiement via npm audit. Seules les librairies activement maintenus et issues des sources reconnues sont intégrées. Les versions sont fixées dans package.json pour éviter les mises à jour automatiques

non contrôlées. Une revue mensuelle des dépendances est recommandée dans le cadre du contrat de maintenance.

VI. Maintenance du site et futures mises à jour

Mises à jour et dépendances

- Revue mensuelle des dépendances npm (npm audit) et application des correctifs de sécurité.
- Mise à jour majeure des librairies (React, React Router, etc.) planifiée une fois par an, après tests de non-régression.

Sauvegardes

- Sauvegardes automatiques de la base PostgreSQL gérées par Railway selon le plan souscrit (fréquence à définir dans le contrat).
- Snapshots du dépôt Git sur GitHub à chaque déploiement.

Déploiement continu

- Déploiements automatiques depuis la branche main du dépôt GitHub vers Railway, sans interruption de service.
- Environnement de staging recommandé pour valider les évolutions avant mise en production.

Evolutions futures

- Toute nouvelle fonctionnalité (version mobile, nouvelle langue, etc.) fera l'objet d'un avenant aux spécifications techniques.
- La durée du support de la version 1.0 est à définir contractuellement, avec une révision majeure recommandée après 18 à 24 mois d'exploitation.